RECITATION 2

# Simple Linear Regression

*sklearn walkthrough · exercises · Chen Hajaj*

# Agenda

**01** **Reminder**
Pre-learning recap — the key equations

**02** **Case Study**
Experience vs. Salary — end-to-end in Python

**03** **Code Walk**
Load → visualize → split → fit → predict → evaluate

**04** **Exercises**
Q1: normalization · Q2: brain vs. body weight

# Reminder — Key Equations from the Lecture

$$y = \beta_0 + \beta_1 x + \varepsilon$$

$$\min \Sigma(y_n - (ax_i + b))^2$$

$$\beta_1 = \Sigma(x_i - \bar{x})(y_i - \bar{y}) \,/\, \Sigma(x_i - \bar{x})^2$$

$$\beta_0 = \bar{y} - \beta_1 \cdot \bar{x}$$

$$\theta := \theta - \alpha \cdot \nabla J(\theta)$$

SLR model · OLS cost · Closed-form β̂ · Intercept · Gradient Descent

# Case Study — Experience vs. Salary

**Problem:** Can we predict a worker's salary from their years of experience?

## Input

Dataset of 30 workers
YearsExperience + Salary

## Output

A linear function:
Salary = $\beta_0 + \beta_1 \cdot$ Experience

## Assumption

Linear correlation (we'll validate with a plot)

Dataset: Salary_Data.csv (30 rows, 2 columns)

# Step 1 — Load & Explore the Data

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

dataset = pd.read_csv('Salary_Data.csv')
print(dataset.head())
print(dataset.describe())
```

## What to check

- How many rows / columns?
- Any missing values?
- Range of each variable
- Does the data look reasonable?

```
X = dataset.iloc[:, :-1].values   # all cols except last → Experience
y = dataset.iloc[:,  1].values    # second col → Salary
```

# Step 2 — Visualise (Always Do This First)

```
plt.scatter(X, y, color='steelblue',
alpha=0.7)
plt.xlabel('Years of Experience')
plt.ylabel('Salary ($)')
plt.title('Experience vs Salary')
plt.tight_layout()
plt.show()
```


**See a line?** Linear regression is justified

**See a curve?** Try polynomial regression

**See a cloud?** No strong relationship — investigate

# Step 3 — Train / Test Split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size = 0.33,    # 33% held out for testing
    random_state = 42     # reproducible split
)
```

| | |
|---|---|
| **X_train / y_train** | ~20 samples — used to fit the model |
| **X_test / y_test** | ~10 samples — never seen during training |
| **random_state=42** | Fix the seed so results are reproducible |

# Step 4 — Fit the Model

```
from sklearn.linear_model import LinearRegression

regressor = LinearRegression()
regressor.fit(X_train, y_train)

# Inspect what the model learned:
print('β₀ (intercept):', regressor.intercept_)    # ≈ 25,792
print('β₁ (slope):    ', regressor.coef_[0])      # ≈  9,450
```

**Salary = 25,792 + 9,450 × Experience**

# Step 5 — Predict & Evaluate

```
y_pred = regressor.predict(X_test)

# Residuals
error = y_test - y_pred

# Key metrics
from sklearn.metrics import
mean_squared_error, r2_score
print('MSE :', mean_squared_error(y_test,
y_pred))
print('R²  :', r2_score(y_test, y_pred))
```

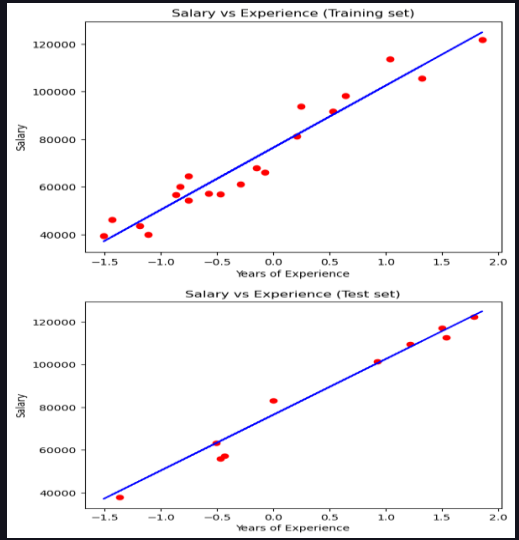
## Metrics

**MSE** Mean squared error — lower is better
**RMSE** √MSE — in original units (dollars)
**$R^2$** % variance explained — 1.0 is perfect

*Rule of thumb: $R^2 > 0.9$ is excellent for a single predictor.*

# Step 6 — Visualise the Fit

```
# Training set
plt.scatter(X_train, y_train,
color='steelblue', label='Train')
plt.plot(X_train, regressor.predict(X_train),
color='tomato', lw=2)
# Test set (same line, new dots)
plt.scatter(X_test, y_test, label='Test')
plt.legend()
plt.show()
```




- We plot the TRAINING line on both charts — same fitted model
- Test dots scattered near the line = good generalisation
- Test dots far from line = model may not generalise

# Full Pipeline at a Glance

**1** **Load**

`pd.read_csv()`

**2** **Explore**

`scatter plot`

**3** **Split**

`train_test_split()`

**4** **Fit**

`regressor.fit(X_train, y_train)`

**5** **Predict**

`regressor.predict(X_test)`

**6** **Evaluate**

`MSE, R²`

# — Exercises

*Apply what you learned*

# Q1 Exercise

דני נזכר שחשוב לנרמל את המשתנים הבלתי תלויים בבעיות רגרסיה.
חיזרו לקוד והוסיפו נרמול (StandardScaler) לפני שלב האימון.

השוו את משוואת הישר שקיבלתם עם הנרמול ובלעדיו — האם היא השתנתה?

א. מה היתרונות של נרמול המשתנים?

ב. מה החסרונות / מתי לא כדאי לנרמל?

# Q1 — Code Scaffold

```
from sklearn.preprocessing import StandardScaler


sc_X = StandardScaler()
X_train_scaled = sc_X.fit_transform(X_train)
X_test_scaled  = sc_X.transform(X_test)      # use SAME scaler!


# Refit with scaled data
regressor2 = LinearRegression()
regressor2.fit(X_train_scaled, y_train)
print('β₀:', regressor2.intercept_)
print('β₁:', regressor2.coef_[0])
```

Note: $\beta_1$ will change (now in units of std-devs), but predictions on new data stay the same if you scale consistently.

# Q2 Exercise

יוסי רצה לבדוק האם יש קשר בין משקל ראש לבין משקל גוף של יונקים.
אסף נתונים מ 62-מינים שונים — הקובץ q2.csv :

בנו מודל רגרסיה לינארית ובדקו:

א. ייצרו פונקציית חיזוי (fit + predict)

ב. מה יהיה משקל גוף של יונק עם משקל ראש של 30ק"ג?

# Q2 — Getting Started

```
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

df = pd.read_csv('q2.csv')
X = df[['brain_weight']].values   # independent
y = df['body_weight'].values      # dependent

# Split, fit, predict — same pipeline as before!
# Expected: β₀ ≈ 33.6,  β₁ ≈ 1.745
# Prediction for brain=30 kg:  ŷ ≈ 85.98 kg
```

- Always plot brain_weight vs body_weight first — check for outliers (whales!)
- Consider log-transforming both variables — mammal weights span many orders of magnitude

# See you next week!

Machine Learning · Recitation 2 · Simple Linear Regression
Chen Hajaj